SIMATS ENGINEERING

ASSESSMENT TOOL – 1

Course Name	Internet Programming
Course Code	CSA4304
Course Outcome	CO-1
Assessment Tool	Application-Based Questions
Total Marks	40%

Name: P.RevanthReddy

Register No.: 192372369

Date: 3/10/2026

1. Online Symposium Portal

Given HTML Structure:

```
<form action="/register" method="POST">
    <input type="text" name="name">
    <input type="email" name="email">
    <button>Submit</button>
</form>
```

1. Identify appropriate HTML5 semantic elements missing in the structure.

The given form contains only a <form> element and input controls. For a properly structured HTML5 webpage, several semantic elements can be added:

- <header> – represents the header of the symposium portal.
- <nav> – provides navigation links.
- <main> – contains the main content of the page.
- <section> – groups related content such as registration.
- <footer> – contains footer information.
- <label> – provides accessible names for form controls.
- <fieldset> and <legend> – can group related form fields.

These elements improve the structure, readability, accessibility, and maintainability of the webpage.

2. Modify the structure to make it standards-compliant.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Online Symposium Registration</title>
</head>
<body>
<header>
    <h1>College Online Symposium</h1>
</header>
```

```
<nav aria-label="Main Navigation">
    <a href="/">Home</a>
    <a href="/events">Events</a>
    <a href="/contact">Contact</a>
</nav>

<main>
    <section>
        <h2>Student Registration</h2>
        <form action="/register" method="POST">
            <fieldset>
                <legend>Registration Details</legend>

                <div>
                    <label for="name">Full Name:</label>
                    <input type="text" id="name" name="name"
                           required autocomplete="name">
                </div>

                <div>
                    <label for="email">Email Address:</label>
                    <input type="email" id="email" name="email"
                           required autocomplete="email">
                </div>

                <button type="submit">Submit</button>
            </fieldset>
        </form>
    </section>
</main>

<footer>
    <p>&copy; 2026 College Symposium</p>
</footer>
</body>
</html>
```

3. Interpret the HTTP request generated during form submission.

When the student clicks the Submit button, the browser collects the values entered into the form and sends them to the server. Because the form uses method="POST", the browser generates an HTTP POST request.

```
POST /register HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

name=Revanth&email=revanth%40example.com
```

POST specifies that data is being submitted to the server, /register identifies the server endpoint, HTTP headers provide request information, and the request body contains the submitted form data.

4. Analyze the HTTP response cycle from server to browser.

- The browser sends the POST request to /register.
- The web server receives the request.
- The server processes the submitted name and email.

- The server performs required validation and registration operations.
- The server generates an HTTP response.
- The browser receives the response.
- The browser interprets the response and displays the resulting webpage.

The communication follows: Browser → HTTP Request → Server → Processing → HTTP Response → Browser → Webpage.

```
HTTP/1.1 200 OK
Content-Type: text/html

<html>
<body>
    <h1>Registration Successful</h1>
</body>
</html>
```

5. Suggest improvements for usability and accessibility.

- Add <label> elements for every input field.
- Use for attributes that match the input id.
- Mark important fields as required.
- Use appropriate input types such as email.
- Provide meaningful error messages.
- Use keyboard-accessible controls.
- Provide visible focus indicators.
- Use semantic HTML5 elements.
- Provide clear instructions for completing the form.
- Use sufficient colour contrast.
- Add appropriate autocomplete attributes.
- Make the page responsive for mobile devices.

2. Cross-Browser Compatibility Issue

The given webpage contains incorrect nesting and missing closing tags.

```
<html>
<head>
    <title>Company</title>
</head>
<body>
    <div>
        <h1>Welcome
        <p>About us
    </div>
</body>
</html>
```

1. Identify improper nesting and missing closing tags.

The <h1> and elements are not properly closed. They should be written as <h1>Welcome</h1> and About us . The original structure therefore has incomplete markup.

2. Analyze how different browsers may interpret this structure.

Different browsers have HTML error-recovery mechanisms. When markup is incomplete or incorrectly nested, browsers may attempt to automatically correct the document structure. As a result, the DOM created by the browser may differ from the author's intended structure, CSS selectors may behave unexpectedly, page layout may become inconsistent, and JavaScript that depends on a particular DOM structure may fail or behave differently.

3. Identify missing semantic elements.

- <header> – for the company heading.
- <nav> – for navigation links if available.
- <main> – for the primary page content.
- <section> – for a logically grouped section.
- <footer> – for footer information.

4. Provide a corrected W3C-compliant HTML structure.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Company</title>
</head>
<body>
<header>
    <h1>Welcome</h1>
</header>

<main>
    <section>
        <h2>About Us</h2>
        <p>About us</p>
    </section>
</main>

<footer>
    <p>&copy; 2026 Company</p>
</footer>
</body>
</html>
```

5. Suggest techniques to ensure cross-browser compatibility.

- Use valid HTML5 syntax.
- Properly close and nest elements.
- Use <!DOCTYPE html>.
- Use standard CSS properties.
- Avoid browser-specific or obsolete HTML features.
- Use responsive CSS.
- Test the website in Chrome, Firefox, Edge, and Safari.
- Use browser developer tools to identify errors.
- Validate HTML and CSS using standards-compliance validators.

- Use feature detection when browser-specific functionality is required.

3. Form Submission Failure

Given HTML Structure:

```
<form action="/submit" method="GET">
    <input type="text" name="username">
    <input type="password" name="password">
    <button type="button">Login</button>
</form>
```

1. Identify issues in the form configuration.

There are two important issues. First, the button uses type="button", which does not automatically submit the form. It should use type="submit". Second, GET is inappropriate for password data because submitted values are placed in the URL.

2. Analyze why the HTTP request is not triggered.

The main reason is that the button is defined as <button type="button">. A normal button does not submit its associated form. Changing it to <button type="submit"> causes the browser to submit the form.

3. Interpret the role of GET method in this scenario.

The GET method is designed primarily for retrieving resources or sending non-sensitive query parameters. For example, a search form can use GET and produce a request such as:

```
GET /search?query=internet HTTP/1.1
```

For a login form, GET is inappropriate because the username and password would be included in the URL. Therefore, a login form should normally use POST rather than GET.

4. Propose a corrected implementation.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login</title>
</head>
<body>
<main>
    <section>
        <h1>Login</h1>

        <form action="/submit" method="POST">
            <div>
                <label for="username">Username:</label>
                <input type="text" id="username" name="username"
                       required autocomplete="username">
            </div>

            <div>
                <label for="password">Password:</label>
                <input type="password" id="password" name="password"
                       required autocomplete="current-password">
```

```
            </div>

            <button type="submit">Login</button>
        </form>
    </section>
</main>
</body>
</html>
```

5. Suggest improvements for secure data transmission.

- Use HTTPS rather than HTTP.
- Use the POST method for submitting credentials.
- Never transmit passwords through plain HTTP.
- Validate input on the server side.
- Store passwords securely using strong password hashing.
- Use secure session management.
- Implement appropriate authentication and authorization controls.
- Do not expose passwords in URLs or query strings.
- Use secure cookies for authentication sessions.
- Provide appropriate error handling without revealing sensitive information.

Important: Using POST alone does not encrypt data. HTTPS/TLS is required to protect credentials while they are transmitted over the network.

Conclusion

The three cases demonstrate important Internet Programming concepts: HTML5 semantic structure improves organization and accessibility; proper HTML syntax and nesting improves browser compatibility; HTTP POST requests are appropriate for submitting form data; GET is mainly suitable for retrieving resources and non-sensitive query data; a form requires a submit button to trigger normal form submission; and HTTPS is essential when transmitting sensitive information such as passwords.